

Project Self-Evaluation

1. Reflection on Design vs. Reality

I believe my design document matches the reality quite well. The idea for this app came from my real-life needs. During the winter in Finland, I often need to take various supplements and medications like Vitamin B2, B6, C, D, and fish oil. Each day I need to manage different types and doses, which creates a genuine user requirement.

The three core features I implemented directly align with both the project requirements and the essential functionality a pill reminder app should have. At the very least, I can now personally use it to track my own pill reminders.

One aspect that did not go as originally planned was the timetable. The schedule I had initially envisioned differed from the one I actually followed during the development process. The final version of the timetable is included in the design document or shown below.

2. Time Spent on the Assignment

- **25.2:** Project Design – *2 hours*
- **25.4 – 10.5:** Research and Programming – *2 hours + 24 hours*
- **23.5 – 30.5:** Debugging and Documentation – *15 hours*

To be honest, I encountered quite a few difficulties while generating Javadoc. Since I used several third-party libraries, I faced many path-related issues. In the end, I decided to switch to Dokka to generate HTML documentation, which worked better for my Java Android project.

3. What I Learned

- **Design is key.** It's not only about UI design, but also about determining how many classes are needed and what their responsibilities are. Poor design decisions forced me to rewrite many files. For instance, I had to redesign my database structure three times. Each time, I needed to update related code across the project, which introduced errors and consumed a lot of time. I also had to split my `DatabaseHelper` class into two separate classes late in development, which again required changes to multiple method calls and references.
- **Learn to think iteratively.** I had ideas for features when I designed this App, but that doesn't mean I needed to build everything at once. Adding one feature at a time helped keep the app stable and improved my understanding of the overall architecture.
- **Deepened my Android development skills.** UI design is fun, but writing functional code is quite challenging. While my app doesn't involve interactive graphics like drawing, calculating, moving objects, or collision detection like in our exercises, I did learn:
 - How to generate charts
 - How to use the date picker and time picker
 - How to implement notification alerts
 - How to work with a local SQLite database

Overall, this project gave me both practical development experience and insight into the importance of thoughtful planning and structure in app development.